

ESP_TIMER

ESP_TIMER

- 1. ESP_TIMER Overview
- 2. Core Concepts
 - 2.1 esp_timer vs Hardware Timer
 - 2.2 Key APIs
 - 2.3 volatile for Cross-Task Variables
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Global Variables
 - 5.2 Timer Callback Function
 - 5.3 setup() — Create and Start Timer
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Expected Behavior
 - 6.3 Verification Methods
- 7. Common Issues

1. ESP_TIMER Overview

ESP_TIMER is a high-precision software timer API provided by ESP-IDF, implemented based on a 64-bit hardware timer with microsecond resolution. Callbacks are executed in a separate Timer Task (not ISR context), allowing the use of `Serial.print`, which is much more convenient than hardware timer ISRs. This experiment creates a periodic timer that triggers every second, printing the trigger count in the callback.

Typical Application Scenarios:

Application Type	Example
Periodic Sampling	Timed reading of temperature/humidity sensors, battery voltage inspection
Communication Heartbeat	MQTT Keep-Alive, Bluetooth heartbeat, CAN bus node monitoring
Watchdog Feeding	Task-level timed feeding to prevent hardware WDT false resets
LED Indication	Timed blinking, waterfall light rhythm control
Timeout Detection	Communication response timeout, sensor data update timeout

2. Core Concepts

2.1 esp_timer vs Hardware Timer

Feature	esp_timer	Hardware Timer (hw_timer_t)
Callback Context	RTOS Timer Task	ISR
Can Use <code>Serial.print</code>	Yes	No
Timing Precision	Microsecond-level (with RTOS scheduling jitter)	Microsecond-level (deterministic delay)
Auto-reload	<code>esp_timer_start_periodic</code> automatic	<code>timerAlarm</code> <code>auto_reload=true</code>
Use Case	Most application timing needs	Jitter-sensitive precise timing

2.2 Key APIs

`esp_timer_create(&args, &handle)` — Creates a timer.

```
const esp_timer_create_args_t timer_args = {
    .callback = esp_timer_callback,      // callback function pointer
    .name = "Esp_Timer",                 // Debug name
    // .dispatch_method defaults to ESP_TIMER_TASK → callback runs in timer task
};
esp_timer_handle_t timer_handle;
esp_timer_create(&timer_args, &timer_handle);
```

`esp_timer_start_periodic(handle, period_us)` — Starts a periodic timer. `period_us` is in microseconds, `1000000` = 1 second. The timer can be stopped at any time using `esp_timer_stop()` after creation.

2.3 volatile for Cross-Task Variables

The callback executes in the Timer Task, while `loop()` executes in the Arduino Task. Shared variables between different tasks require the `volatile` modifier, otherwise the compiler may cache the variable in each task's CPU registers, causing stale values to be read.

3. Hardware Dependencies

This project is a pure software experiment with no external hardware required. Only a USB cable is needed to connect to the serial port for viewing output.

4. Project Architecture and Flow

```
setup()
├─ Serial.begin(115200)           // Initialize serial port
├─ esp_timer_create(&args, &handle) // Create timer
└─ esp_timer_start_periodic(handle, 1000000) // Start: 1s period

loop()                             // Empty loop
└─ (Timer callback runs independently in background Timer Task)

[Timer Task Background]
├─ esp_timer_callback()           // Called automatically every second
│   ├─ trigger_count++           // Increment counter
│   └─ Serial.printf(...)        // Print trigger info
```

5. Source Code Details

Complete source code: [Source Code](#) → [Arduino](#) → [1.Basic Peripherals](#) → [4.ESP_TIMER](#) → [4.ESP_TIMER.ino](#)

5.1 Global Variables

`trigger_count` is written in the callback and read in `loop()` (if needed). Cross-task sharing requires `volatile`.

```
volatile uint32_t trigger_count = 0; // Timer trigger count counter
```

If `loop()` doesn't need to read this variable, `volatile` can be omitted, but keeping it is good practice for future extensibility.

5.2 Timer Callback Function

The callback runs in the Timer Task, not ISR, so `Serial.printf` is safe to use.

```
void esp_timer_callback(void* arg) {
    trigger_count++;
    Serial.printf("Trigger count: %u | Current time: %lu ms\n", trigger_count, millis());
}
```

5.3 `setup()` — Create and Start Timer

First configure the `esp_timer_create_args_t` structure, then create the timer, and finally start it.

```
void setup() {
    Serial.begin(115200);
    Serial.println("ESP_TIMER Started (1000ms Period)");

    const esp_timer_create_args_t timer_args = {
```

```

        .callback = esp_timer_callback,
        .name = "Esp_Timer"
    };

    esp_timer_handle_t timer_handle;
    esp_timer_create(&timer_args, &timer_handle);
    esp_timer_start_periodic(timer_handle, 1000000); // 1,000,000 us = 1s
}

void loop() {} // Empty loop – all work done by timer callback

```

`loop()` is empty because there is no polling logic needed. The timer runs independently in the background with callbacks triggering automatically, which is the perfect example of event-driven programming.

6. Operation Flow

6.1 Build and Flash

Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

6.2 Expected Behavior

Open the **Serial Monitor** (115200 bps) and one line is printed every second:

```

=====
ESP_TIMER Started (1000ms Period)
=====
Trigger count: 1 | Current time: 1026 ms
Trigger count: 2 | Current time: 2026 ms
Trigger count: 3 | Current time: 3026 ms
Trigger count: 4 | Current time: 4026 ms
...

```

```

ESP-ROM: esp32s3-20210327
Build: Mar 27 2021
rst: 0x1 (POWERON), boot: 0x28 (SPI_FAST_FLASH_BOOT)
SPIWP: 0xee
mode: DIO, clock div: 1
load: 0x3fce2820, len: 0x10cc
load: 0x403c8700, len: 0xc2c
load: 0x403cb700, len: 0x30c0
entry 0x403c88b8
E (88) esp_core_dump_flash: Incorrect size of core dump image: -2147418112
=====
ESP_TIMER Started (1000ms Period)
=====
Trigger count: 1 | Current time: 1026 ms
Trigger count: 2 | Current time: 2026 ms
Trigger count: 3 | Current time: 3026 ms
Trigger count: 4 | Current time: 4026 ms
Trigger count: 5 | Current time: 5026 ms
Trigger count: 6 | Current time: 6026 ms
Trigger count: 7 | Current time: 7026 ms
Trigger count: 8 | Current time: 8026 ms
Trigger count: 9 | Current time: 9026 ms
Trigger count: 10 | Current time: 10026 ms
Trigger count: 11 | Current time: 11026 ms
Trigger count: 12 | Current time: 12026 ms
Trigger count: 13 | Current time: 13026 ms

```

6.3 Verification Methods

Action	Expected Result
Observe serial port after power on	One line per second, <code>trigger_count</code> increments
Change <code>1000000</code> to <code>500000</code>	Triggers every 500ms

7. Common Issues

Symptom	Cause	Solution
No serial output	<code>loop()</code> is empty but Serial not initialized	Verify <code>Serial.begin(115200)</code>
Callback trigger interval inaccurate	RTOS scheduling jitter (normal)	Use hardware timer for extreme precision requirements
Compilation error <code>esp_timer.h</code> not found	Missing IDF component	No additional action needed, Arduino ESP32 Core includes it by default